SAÉ S3.A.01 - Développement d’une application

Autour des fractales

IUT de Lens - Département Informatique

2026-2027

Table des matières

Phase 1 - Création d'une bibliothèque de génération de fractales **1**
Organisation du travail . . . 2
Groupes de travail . . . 2
Encadrants . . . 2
Organisation de votre projet . . . 2
Évaluation individuelle . . . 3
Évaluation . . . 4

***Sprint 0* : Analyse du besoin** **5**
Contexte . . . 5
Fonctionnalités attendues . . . 5
Le plan complexe . . . 6
Du pixel au nombre complexe . . . 6
Fractales et suites de nombres complexes . . . 7
Dessin de la fractale . . . 7
Images statiques et animées . . . 7
Configuration du générateur de fractales . . . 8
Tableau récapitulatif des fonctionnalités . . . 8

***Sprint 1* : Représentation des nombres complexes** **11**
Travail préliminaire - *Fork* du dépôt *GitLab* . . . 11
But du *sprint* . . . 11
Description technique du *sprint* . . . 12
Opérations sur les nombres complexes . . . 12
Représentation des nombres complexes dans le plan . . . 13
Suites de nombres complexes : cas de la suite de Julia . . . 13
Attendus de ce *sprint* . . . 14
Validations techniques à réaliser en cours de *sprint* . . . 15
Démonstration à présenter à la fin du *sprint* . . . 15

***Sprint 2* : Première génération de fractale** **17**
But du *sprint* . . . 17
Description technique du *sprint* . . . 18
Mise à l'échelle du plan complexe . . . 18
Choix d'une couleur pour un pixel . . . 18
Dessin de l'image de fractale . . . 19
Génération de l'image de fractale . . . 19
Programme principal . . . 20
Attendus de ce *sprint* . . . 20
Validations techniques à réaliser en cours de *sprint* . . . 20
Démonstration à présenter à la fin du *sprint* . . . 21

***Sprint 3* : Variations sur la suite de Julia** **23**
But du *sprint* . . . 23
Description technique du *sprint* . . . 23
Modification du centre de l'image . . . 23
Changement de la palette de couleurs . . . 24
Attendus de ce *sprint* . . . 26
Validations techniques à réaliser en cours de *sprint* . . . 26
Démonstration à présenter à la fin du *sprint* . . . 26

***Sprint 4* : Applications à d'autres suites complexes** **29**
But du *sprint* . . . 29
Description technique du *sprint* . . . 30
Suite de Mandelbrot . . . 30
Suites Multijulia et Multibrot . . . 31
Suites du *Burning Ship* et du *Tricorn* . . . 31
Suites définies à l'aide d'expressions plus générales . . . 31
Attendus de ce *sprint* . . . 32
Validations techniques à réaliser en cours de *sprint* . . . 32
Démonstration à présenter à la fin du *sprint* . . . 32

***Sprint 5* : Configuration du générateur** **35**
But du *sprint* . . . 35
Description technique du *sprint* . . . 35
Encapsulation des suiteset stratégies dans des énumérations . . . 35
Implémentation du *builder* . . . 36
Attendus de ce *sprint* . . . 36
Validations techniques à réaliser en cours de *sprint* . . . 37

Démonstration à présenter à la fin du *sprint* 37

***Sprint 6* : Génération d'une animation** **39**
But du *sprint* 39
Description technique du *sprint* 39
Attendus de ce *sprint* 41
Validations techniques à réaliser en cours de *sprint* 41
Démonstration à présenter à la fin du *sprint* 42

***Sprint 7* : Gestion de la ligne de commande et finalisation du projet** **43**
But du *sprint* 43
Description technique du *sprint* 43
Gestion de la ligne de commande 43
Bonnes pratiques de programmation 44
Attendus de ce *sprint* 45
Validation technique à réaliser en cours de *sprint* 45
Démonstration à présenter à la fin du *sprint* 45

Final de la SAÉ - Concours de fractales **47**
Règles de soumission 47
Règles du vote 47

Phase 1 - Création d'une bibliothèque de génération de fractales

Félicitations ! Votre équipe a été choisie par un lycée partenaire pour développer la bibliothèque logicielle *FractaLens* visant à faire découvrir des propriétés mathématiques à ses élèves de manière ludique. Cette bibliothèque devra être capable de générer une large variété d'images de fractales mathématiques, et de les sauvegarder dans différents formats.



Figure 1 – Logo de *FractaLens*.

Organisation du travail

Cette première semaine de SAÉ vise à évaluer votre capacité à développer en équipe une application informatique de façon efficace et qualitative.

Pour rappel, une SAÉ (Situation d'Apprentissage et d'Évaluation) doit vous amener à acquérir par vous-mêmes de nouvelles connaissances en autonomie. Il est donc normal que l'on vous demande un travail que vous n'avez jamais effectué, et que vous ne soyez pas encadrés tout au long des séances. Vous aurez la possibilité de poser des questions sur l'équipe Mattermost dédiée à la SAÉ, et lors des passages des enseignants.

Vous devez montrer que vous avez acquis les compétences que l'on attend de vous en donnant les raisonnements qui justifient vos choix, notamment lors des démonstrations techniques intermédiaires.

En aucun cas votre travail ne pourra être similaire à celui d'un autre groupe.

Groupes de travail

Vous travaillerez en groupe de 4 dans cette SAÉ, en veillant à ce qu'**aucune équipe ne soit composée de plus d'un·e doublant·e**. Vous avez normalement déjà formé vos groupes via cette activité du cours *Moodle*. Vous pourrez travailler sur vos machines ou sur les machines des salles TP qui vous ont été attribuées.

Encadrants

Les encadrants de cette première phase de la SAÉ sont :

— Fred Hémery
— Romain Wallon

Organisation de votre projet

Votre projet devra être partagé avec les membres de votre groupe et les enseignants qui corrigeront cette SAÉ (Fred Hémery et Romain Wallon) au travers d'un projet *GitLab* correspondant à un *fork* d'un dépôt qui vous sera fourni en temps voulu.

Dans ce projet, vous devrez créer les différents jalons et tickets correspondant aux différentes tâches à réaliser, que vous assignerez entre les membres de votre groupe. Vous devrez planifier ces différentes tâches en suivant la **méthodologie Agile**, en respectant les *sprints* qui vous seront imposés. Vous

devrez vous organiser de manière à ce que le travail soit **réparti équitablement** entre les différents membres du groupe, et que chaque tâche soit **terminée dans les temps**. Une mini-démonstration de votre application sera réalisée à la fin de chaque *sprint*. Au cours des *sprints*, il vous sera par ailleurs demandé de faire valider certains points techniques par les enseignants : veillez à le faire dès que possible pour permettre des passages fluides. **Ces démonstrations techniques sont obligatoires, et vont constituer une partie de votre évaluation individuelle et de groupe.**

Au cours du développement de votre projet, vous veillerez à faire régulièrement des ***commits* atomiques**, en mentionnant dans chaque message de *commit* la **référence du ticket** correspondant à la tâche en cours de réalisation.

Chaque fonctionnalité devra être réalisée sur une branche dédiée, et fusionnée sur une branche `dev` une fois terminée, puis sur la branche `main` une fois les fonctionnalités validées à l'issue de la démonstration. Les fusions sur la branche `main` devront correspondre exactement aux **versions stables** publiées **à chaque fin de *sprint*** et présentées lors des démonstrations. Un **tag** doit être créé à chaque fin de *sprint* sur la branche `main`, afin de garder une trace de la dernière version stable produite à la fin de chaque *sprint*.

Évaluation individuelle

Au cours de ce projet, vous allez développer collaborativement une application. Pendant ce développement, **chacun d'entre vous doit mobiliser les** six compétences de la deuxième année de BUT Informatique**. Pour cette première phase, ce sont en particulier les compétences 1, 2 et 6 qui seront concernées. Leurs composantes essentielles sont rappelées ci-dessous :

- **Compétence 1 : Réaliser un développement d'application.** Développer – c'est-à-dire concevoir, coder, tester et intégrer – une solution informatique pour un client en respectant les besoins décrits par le client, en implémentant les algorithmes correspondants aux besoins, en veillant à la qualité du code et à sa documentation, en choisissant les ressources techniques appropriées.
- **Compétence 2 : Optimiser des applications.** Proposer des applications informatiques optimisées en fonction de critères spécifiques (temps d'exécution, précision, consommation de ressources, …) en analysant et en modélisant les attentes du client, en utilisant les algorithmes et structures de données appropriés, en adaptant des modèles de solution, en justifiant ses choix techniques et en validant les résultats.
- **Compétence 6 : Collaborer au sein d'une équipe informatique.** Acquérir, développer et exploiter les aptitudes nécessaires pour travailler efficacement dans une équipe informatique en attribuant des rôles aux membres de son équipe, en utilisant des outils facilitant la communication et le travail collaboratif, en utilisant des technologies de dernière génération, en veillant au respect des contraintes juridiques.

Le sujet de SAÉ qui vous est proposé a été conçu de manière à ce que vous ayez besoin de mobiliser ces compétences. Afin de nous assurer que chaque membre du groupe les a effectivement mobilisées, **chaque membre devra faire valider un point technique dans au moins deux composantes essentielles des compétences 1 et 2 (la compétence 6 sera évaluée sur la base de votre répartition des tâches sur *GitLab*)**. La liste des validations techniques vous est fournie dans la grille d'évaluation ainsi que dans la description des différents *sprints*.

Évaluation

L'évaluation globale du projet se fera sur la base des livrables suivants, qui constitueront la note de groupe de votre projet :

- Le **diagramme de classes prévisionnel** de votre application et la **liste des patrons de conception identifiés**, que vous déposerez sur *Moodle* avant le **lundi 19 octobre à 12h00**.
- Le **diagramme de classes final** de votre application et la **liste des fonctionnalités implémentées**, que vous veillerez à **compléter à chaque *sprint*** dans le fichier `README.md` de votre projet.
- Les **validations techniques** réalisées en cours de *sprint* et précisées dans la description de chaque *sprint*.
- Les **démonstrations** réalisées à chaque fin de *sprint*, sous la forme de tests à exécuter et/ou d'une ou plusieurs images de fractale que vous aurez générée(s) avec votre bibliothèque en respectant les contraintes imposées par le sujet.
- Le code source **écrit, testé et documenté en anglais** de votre application sur *Gitlab*, dont vous déposerez le lien sur *Moodle*. Vous ajouterez pour cela les correcteurs de cette SAÉ comme *reporters* sur votre projet, **avec comme date limite le 31 mars 2027**.

Une évaluation individuelle sera réalisée sur la base des validations techniques réalisées pendant les différents *sprints*.

Sprint 0 : Analyse du besoin

Au cours de ce *sprint*, vous devez analyser le besoin exprimé par le client au sujet de votre bibliothèque de génération de fractales, en considérant la description détaillée des fonctionnalités attendues donnée ci-dessous. Vous devez ensuite réaliser le diagramme de classes de votre bibliothèque, en intégrant un **maximum de patrons de conception**. Dans ce diagramme, vous veillerez tout particulièrement à **représenter les relations entre les classes** en respectant les règles de modélisation UML (composition, agrégation, héritage, implémentation, *etc.*). **Vous n'êtes pas obligés de préciser les attributs ou méthodes dans ces classes, à la condition que le rôle de chaque classe soit suffisamment explicite (en particulier, essayez de trouver des noms significatifs).** Vous vous assurerez que ce diagramme **reste lisible** : si vous estimez qu'il est trop complexe, vous pouvez le scinder en plusieurs diagrammes, chacun représentant un aspect particulier de votre bibliothèque.

Vous préciserez également dans le tableau fourni sur *Moodle* et rappelé ci-dessous le patron choisi pour chacune des fonctionnalités attendues (le cas échéant). Ces deux livrables (diagramme de classes et tableau récapitulatif des fonctionnalités) seront à rendre sur *Moodle* avant la fin du *sprint*.

Contexte

Dans ce projet, vous devez concevoir et implémenter une bibliothèque logicielle de génération de fractales en *Java*, en utilisant des suites de nombres complexes permettant de produire de jolies fractales. Vous pouvez retrouver tous les détails mathématiques derrière l'utilisation de ces suites sur cette page, en plus de la description donnée ci-dessous. Le moment venu, le détail des *sprints* vous donnera tous les détails nécessaires à l'implémentation des différentes classes et méthodes de votre bibliothèque.

Fonctionnalités attendues

Les fractales que nous allons considérer dans ce projet sont des ensembles de points du plan complexe. Un nombre complexe est un nombre z qui peut s'écrire sous la forme $z = a + ib$, où a et b sont des nombres réels et i est un nombre dit *imaginaire* tel que $i^2 = -1$. Le nombre a est appelé *partie*

réelle de z (noté $\mathfrak{Re}(z)$) et le nombre b est appelé *partie imaginaire* de z (noté $\mathfrak{Im}(z)$). Plusieurs opérations classiques sont définies sur les nombres complexes, notamment l'addition, la soustraction, la multiplication et la division. Il est également possible de calculer le module d'un nombre complexe (défini comme la distance de ce nombre à 0) ainsi que son conjugué (le conjugué de $z = a + ib$ est le nombre $\overline{z} = a - ib$).

Le plan complexe

Un nombre complexe $z = a + ib$ peut être vu non seulement comme une valeur algébrique, mais aussi comme un point géométrique. Il est alors possible d'associer à z le point de coordonnées (a, b) dans un plan à deux dimensions appelé *plan complexe* : l'axe horizontal (abscisse) correspond à la partie réelle a, et l'axe vertical (ordonnée) à la partie imaginaire b. Ce plan est donc équivalent au plan cartésien $\mathbb{R}^2$, et c'est sur ce support que nous allons dessiner les fractales : chaque pixel d'une image sera mis en correspondance avec un point du plan complexe, donc avec un nombre complexe.

Du pixel au nombre complexe

Pour établir cette correspondance, il faut une règle de conversion entre les coordonnées d'un pixel dans l'image et les coordonnées du point associé dans le plan complexe. Si l'image comporte w pixels par ligne et h pixels par colonne, le pixel situé à la ligne r et à la colonne c est associé au nombre complexe $z = a + ib$ tel que :

$$a = c - \frac{w}{2}$$

$$b = \frac{h}{2} - r$$

Avec cette formule, le plan complexe obtenu est par défaut centré sur le point 0, et s'étend de $-\frac{w}{2}$ à $+\frac{w}{2}$ en abscisse et de $-\frac{h}{2}$ à $+\frac{h}{2}$ en ordonnée. Pour explorer une fractale, il est souvent utile de se déplacer dans le plan complexe (translation) ou de « zoomer » sur une zone particulière (changement d'échelle) :

- **Translation** : pour centrer le plan sur un point $z_c = x + iy$, il suffit d'ajouter x à la partie réelle et y à la partie imaginaire de chaque nombre complexe obtenu par la formule précédente (autrement dit, d'ajouter z_c à ce nombre complexe).
- **Changement d'échelle** : pour zoomer avec un facteur s, il suffit de multiplier la partie réelle et la partie imaginaire de chaque nombre complexe obtenu par la formule précédente par ce facteur s.

Fractales et suites de nombres complexes

Les fractales que nous considérons sont construites à partir d'une **suite récurrente** de nombres complexes, définie à partir d'un terme initial z_0 et d'une fonction f telle que $z_{n+1} = f(z_n)$. Un point p du plan complexe appartient (ou non) à la fractale selon le comportement de la suite étudiée à partir de $z_0 = p$. Pour cela, on procède de la manière suivante :

- on calcule les termes successifs de cette suite ;
- on s'arrête dès que le module d'un terme dépasse 2 (la suite est alors considérée comme *divergente*) ;
- si ce seuil n'est jamais atteint après un nombre fixé d'itérations, le point p est considéré comme appartenant à la fractale.

Dessin de la fractale

Le nombre d'itérations nécessaires avant de dépasser le seuil de divergence (ou le fait de ne jamais l'atteindre) va nous permettre de déterminer la couleur du pixel correspondant à ce point :

- si le point appartient à la fractale (la suite n'a pas divergé), le pixel est en général coloré en **noir** ;
- sinon, le nombre d'itérations effectuées avant de détecter la divergence sert à déterminer la couleur du pixel — en général, plus ce nombre est grand, plus la couleur est sombre.

Il existe plusieurs façons de transformer un nombre d'itérations en couleur :

- **niveaux de gris** : la composante de gris diminue quand le nombre d'itérations augmente ;
- **niveaux de rouge, vert ou bleu** : on peut obtenir ces teintes à partir d'un niveau de gris en ne conservant que la composante souhaitée (rouge, vert ou bleu) et en mettant les autres à 0 ;
- **couleurs inversées** : on prend le complémentaire de la couleur obtenue par l'une des méthodes précédentes ;
- **dégradé entre deux couleurs** fixées ;
- **palette « à la main »**, c'est-à-dire une table qui associe une couleur prédéterminée à chaque nombre d'itérations possible.

Afin d'affecter une couleur à un pixel, nous utiliserons la classe `BufferedImage` fournie dans la bibliothèque standard de *Java*, en veillant à simplifier son interface pour que celle-ci soit plus simple à manipuler dans le cadre de ce projet.

Images statiques et animées

Le générateur de fractales doit être capable de produire des images statiques ou animées.

Une image statique correspond à une seule image de la fractale, générée en suivant les règles précédentes.

Pour réaliser une image animée, il faut générer une série d'images de fractales, en faisant évoluer certaines des caractéristiques de la fractale, par exemple :

— en appliquant une translation progressive sur le plan complexe sur lequel la fractale est dessinée, afin de se déplacer dans la fractale ;
— en appliquant un changement d'échelle progressif sur le plan complexe sur lequel la fractale est dessinée, afin de « zoomer » sur des détails de la fractale ;
— en faisant varier la suite de nombres complexes utilisée pour générer la fractale, afin de produire des images différentes.

La production de ces images successives servira à la production d'une image GIF animée réunissant toutes les images produites.

Configuration du générateur de fractales

Avec toutes ces fonctionnalités, il doit être possible de configurer un « générateur de fractales » capable de produire des images de fractales à partir d'une suite de nombres complexes, d'un nombre d'itérations maximal, d'une palette de couleurs, du centre du plan complexe considéré et de l'échelle de la représentation, et de produire soit une image statique, soit une image animée. Cela doit à la fois pouvoir être fait de manière programmatique, mais également depuis la ligne de commande, en utilisant des arguments passés en paramètre à l'exécution du programme.

Tableau récapitulatif des fonctionnalités

Afin de vous assurer de ne pas manquer de fonctionnalités, le tableau suivant récapitule toutes les fonctionnalités attendues pour votre bibliothèque de génération de fractales. Vous devez compléter ce tableau (disponible sous la forme d'un tableur sur *Moodle*) en indiquant pour chaque fonctionnalité le ou les patrons de conception que vous avez choisi d'utiliser pour l'implémenter. Si vous ne pensez pas utiliser de patron de conception pour une fonctionnalité donnée, vous pouvez laisser la case correspondante vide.

Fonctionnalité	Patron(s) de conception (?)
Nombre complexe	
Plan complexe par défaut	

Fonctionnalité	Patron(s) de conception (?)
Plan complexe mis à l'échelle	
Plan complexe translaté	
Suite de nombres complexes	
Parcours des termes d'une suite de nombres complexes	
Palette de couleurs	
Palette de niveaux de gris	
Palette de niveaux de rouge/vert/bleu	
Palette de couleurs inversées	
Palette de dégradés	
Palette de couleurs personnalisées	
`BufferedImage` simplifiée	
Génération d'images statiques	
Génération d'images animées avec translation	
Génération d'images animées avec zoom	
Génération d'images animées avec changement de suite	
Configuration du générateur de fractales	
Interface en ligne de commande	

Sprint 1 : Représentation des nombres complexes

Dans ce premier *sprint* de développement, vous devez implémenter les classes permettant de représenter les nombres complexes qui serviront de base pour la génération des fractales à partir de suites complexes.

Travail préliminaire - *Fork* du dépôt *GitLab*

Pour pouvoir développer votre application, vous devez d'abord créer un *fork* de ce dépôt. Vous pouvez aussi en retrouver le lien sur *Moodle*. Ajoutez ensuite les membres de votre groupe au moins comme *developers* dans ce projet, ainsi que les correcteurs de cette SAÉ comme *reporters* sur votre projet, **avec comme date limite le 31 mars 2027**.

Avant de commencer tout développement, vous veillerez à cloner le projet sur votre machine et à mettre à jour le fichier `README.md` de votre projet avec vos informations personnelles. Vous créerez également les différents tickets correspondant aux fonctionnalités à implémenter dans ce *sprint*, toutes rattachées au jalon *Sprint 1*, et à vous les assigner. Assurez-vous de respecter toutes les consignes de développement indiquées dans l'introduction de ce sujet.

But du *sprint*

À l'issue de ce *sprint*, votre bibliothèque doit fournir des implémentations de base pour représenter les nombres complexes, le plan complexe et les suites complexes, afin de passer les tests fournis dans les classes de test `ComplexTest`, `ComplexPlaneTest` et `ComplexSequenceTest`. Vous veillerez à compléter ces classes de manière à ce que les méthodes marquées d'un `TODO` retournent une instance **correctement initialisée** de la (ou des) classe(s) que vous avez implémentée(s) (consultez la documentation et les commentaires de ces méthodes pour plus de détails). De plus, si les tests proposés n'atteignent pas 80% de couverture de code, vous devrez compléter ces tests pour atteindre ce seuil. Vous ne devez en revanche pas modifier les tests existants : s'ils ne passent pas, c'est vraisemblablement parce que votre implémentation est incorrecte.

Description technique du *sprint*

Pour atteindre cet objectif, vous devez tout d'abord implémenter l'interface `IComplex` qui définit les opérations élémentaires sur les nombres complexes. Un nombre complexe est un nombre z qui peut s'écrire sous la forme $z = a + ib$, où a et b sont des nombres réels et i est un nombre dit *imaginaire* tel que $i^2 = -1$. Le nombre a est appelé *partie réelle* de z (noté $\mathfrak{Re}(z)$) et le nombre b est appelé *partie imaginaire* de z (noté $\mathfrak{Im}(z)$).

Opérations sur les nombres complexes

Les opérations arithmétiques usuelles sont étendues aux nombres complexes de manière naturelle, en utilisant les propriétés du nombre imaginaire i et les règles arithmétiques de développement et de factorisation. Ainsi, l'addition et la soustraction de deux nombres complexes $z_1 = a_1 + ib_1$ et $z_2 = a_2 + ib_2$ sont définies par les formules suivantes :

$$z_1 + z_2 = (a_1 + a_2) + i(b_1 + b_2)$$

$$z_1 - z_2 = (a_1 - a_2) + i(b_1 - b_2)$$

La multiplication d'un nombre complexe $z = a + ib$ par un nombre réel r est définie par la formule suivante :

$$r \cdot z = (r \cdot a) + i(r \cdot b)$$

La multiplication de deux nombres complexes $z_1 = a_1 + ib_1$ et $z_2 = a_2 + ib_2$ est définie par la formule suivante :

$$z_1 \cdot z_2 = (a_1 \cdot a_2 - b_1 \cdot b_2) + i(a_1 \cdot b_2 + a_2 \cdot b_1)$$

La division d'un nombre complexe $z_1 = a_1 + ib_1$ par un nombre complexe $z_2 = a_2 + ib_2$ est définie par la formule suivante :

$$\frac{z_1}{z_2} = \frac{(a_1 \cdot a_2 + b_1 \cdot b_2) + i(a_2 \cdot b_1 - a_1 \cdot b_2)}{a_2^2 + b_2^2}$$

Le module d'un nombre complexe $z = a + ib$ est défini comme la distance de ce nombre à 0, et est calculé par la formule suivante :

$$|z| = \sqrt{a^2 + b^2}$$

Le conjugué du nombre complexe z est quant à lui défini par la formule suivante :

$$\overline{z} = a - ib$$

Représentation des nombres complexes dans le plan

Chaque nombre complexe peut être représenté par un point dans le plan complexe, où l'axe horizontal représente la partie réelle et l'axe vertical représente la partie imaginaire. Ainsi, le nombre complexe $z = a + ib$ est représenté par le point de coordonnées (a, b) dans le plan complexe (qui est équivalent au plan cartésien $\mathbb{R}^2$). Pour représenter ce plan complexe, vous devez implémenter l'interface `IComplexPlane`, qui fournit une méthode `asComplex(int row, int column)` permettant de déterminer le nombre complexe associé à un pixel de l'image, donné par sa ligne et sa colonne dans l'image. Cette méthode doit retourner le nombre complexe $z = a + ib$ tel que :

- $a = \texttt{column} - \frac{\texttt{width}}{2}$
- $b = \frac{\texttt{height}}{2} - \texttt{row}$

Notez que `height` et `width` correspondent respectivement au nombre de pixels dans chaque ligne et dans chaque colonne de l'image, et sont des caractéristiques propres à chaque instance de votre classe implémentant l'interface `IComplexPlane`.

Suites de nombres complexes : cas de la suite de Julia

Le plan complexe va nous permettre d'étudier le comportement de plusieurs suites de nombres complexes en certains points de ce plan, afin de pouvoir dessiner des fractales. Nous avons vu que chaque fractale est caractérisée par une fonction f propre, utilisée pour calculer le terme suivant d'une suite de nombres complexes à partir du terme précédent ($z_{n+1} = f(z_n)$). Choisir une fractale revient donc à choisir cette fonction f, le principe de calcul restant le même indépendamment de f. Pour pouvoir représenter n'importe quelle fractale basée sur ce principe, il faut donc pouvoir représenter cette fonction f de manière générique, indépendamment de la formule qu'elle utilise. Pour cela, vous devez également implémenter l'interface `IComplexSequence`, qui fournit une méthode `computeNext(IComplex previous)` permettant de calculer le prochain terme d'une suite complexe à partir du terme précédent. Cette première implémentation devra définir la suite de Julia, donnée par la formule suivante :

$$j_n(z) = j_{n-1}(z)^2 + c$$

Ici, c est un nombre complexe fixé qui caractérise la suite, et $j_0(z) = z$ est le nombre complexe à partir duquel la suite est évaluée, ces deux nombres étant fournis à l'instanciation de la suite. Cette suite sera également caractérisée par un nombre maximal d'itérations à effectuer, fourni lui aussi à l'instanciation de la suite, qui permettra de déterminer si la suite diverge ou non.

Observez que l'interface `IComplexSequence` fournie étend elle-même l'interface `Iterable<IComplex>` de la bibliothèque standard, afin de permettre le parcours de ses termes à l'aide d'un **`for`**`-each`. Pour cela, vous devez donc également implémenter dans une nouvelle classe l'interface `Iterator<IComplex>`. Cette implémentation devra définir un attribut `maxIterations` qui permettra de limiter le nombre d'itérations à considérer, et calculer progressivement les termes de la suite à partir du nombre complexe initial, en utilisant la méthode `computeNext(IComplex previous)` de la suite considérée. **Votre implémentation de l'itérateur doit être indépendante de l'implémentation de `IComplexSequence` considérée**, afin de pouvoir être réutilisée pour d'autres suites complexes. Vous veillerez donc autant que possible à utiliser l'interface plutôt que son implémentation concrète dans votre code.

Rappelez-vous que la méthode `hasNext()` de l'itérateur doit retourner **`false`** dès que le nombre d'itérations effectuées dépasse `maxIterations` ou que le module du nombre complexe obtenu dépasse 2, et que c'est la méthode `next()` qui doit calculer le prochain terme de la suite à partir du précédent, en mettant à jour les attributs appropriés de l'itérateur pour pouvoir calculer le terme suivant lors du prochain appel à cette méthode. **Vous ne devez calculer qu'un seul terme à la fois dans cette méthode !**

Lorsque votre itérateur sera correctement implémenté, vous pourrez l'utiliser pour implémenter la méthode `iterator()` dans votre classe implémentant l'interface `IComplexSequence` pour la suite de Julia.

Attendus de ce *sprint*

- Implémentation de l'interface `IComplex`.
- Implémentation de l'interface `IComplexPlane`.
- Implémentation de l'interface `IComplexSequence` pour la suite de Julia.
- Implémentation de l'itérateur pour les suites.

Validations techniques à réaliser en cours de *sprint*

- Implémentation de l'interface `IComplexPlane`.
- Implémentation de l'interface `IComplexSequence` pour la suite de Julia.
- Implémentation de l'itérateur pour les suites.

Démonstration à présenter à la fin du *sprint*

- Les tests unitaires de la classe `ComplexTest` passent tous.
- Les tests unitaires de la classe `ComplexPlaneTest` passent tous.
- Les tests unitaires de la classe `ComplexSequenceTest` passent tous.
- Couverture des tests supérieure à 80% pour les classes implémentant `IComplex`, `ComplexPlane` et `ComplexSequence`.

Pour exécuter les tests unitaires, vous pouvez utiliser la commande suivante dans un terminal à la racine du projet :

```
1 gradle test
```

Le rapport de couverture des tests sera automatiquement généré dans le fichier `build/reports/jacoco/test/html/index.html` que vous pourrez ouvrir dans votre navigateur pour visualiser les résultats. Ces qui validera le passage de tous les tests unitaires et la couverture de code obtenue à l'issue du *sprint*.

Sprint 2 : Première génération de fractale

Dans ce deuxième *sprint*, votre bibliothèque devra être capable de générer une première image de fractale et de la sauvegarder dans un fichier.

But du *sprint*

À l'issue de ce *sprint*, votre bibliothèque doit être capable de générer une image d'une fractale en niveaux de gris, à partir de la suite complexe de Julia que vous avez implémentée dans le *sprint* précédent. Vous devrez en particulier pouvoir générer l'image de la fractale de Julia pour $c = -0.4 + 0.6i$, donnée ci-dessous.

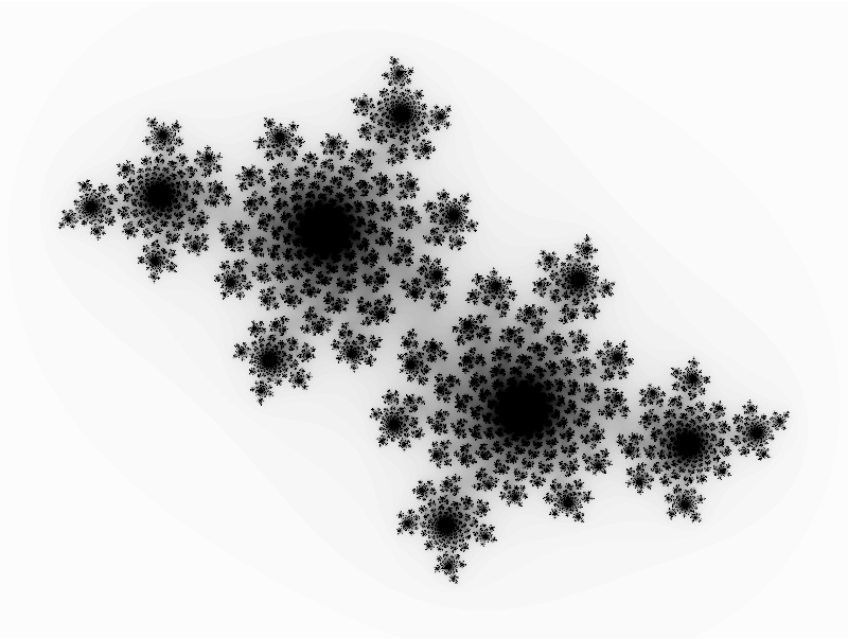


Figure 1 – Fractale de Julia pour $c = -0.4 + 0.6i$ en niveau de gris.

Description technique du *sprint*

La génération d'une fractale consiste à évaluer, pour chaque pixel de l'image, le nombre d'itérations nécessaires pour déterminer si la suite complexe dont le premier terme est le nombre complexe correspondant à ce pixel diverge ou non. Nous allons donc dans un premier temps devoir exploiter le plan complexe que vous avez implémenté dans le *sprint* précédent, afin de déterminer le nombre complexe correspondant à chaque pixel de l'image.

Mise à l'échelle du plan complexe

La fractale de Julia que vous allez générer au cours de ce *sprint* s'étend sur une toute petite portion du plan complexe de base, où un pixel correspond à une unité dans le plan. Vous devez donc définir un décorateur de `IComplexPlane` qui permettra de « zoomer », en multipliant les coordonnées du pixel obtenu par le plan décoré par un facteur réel, propre à l'instance du décorateur.

> **À noter** : lors de la démonstration, il vous sera demandé de générer l'image en appliquant un zoom de 0.004 sur le plan complexe de base. Bien que cela puisse sembler contre-intuitif, un zoom de 0.004 correspond à bien à un agrandissement de l'image, car un pixel de l'image correspond alors à 0.004 unité dans le plan complexe de base. Il faut donc maintenant 250 pixels pour représenter une unité dans le plan complexe de base, ce qui correspond bien à un zoom sur l'image de 250 fois. Si vous préférez penser en termes de « zoom » au sens habituel, vous pouvez à la place appliquer un facteur de zoom entier n et multiplier les coordonnées du pixel par $1/n$ dans votre décorateur, mais il vous faudra adapter le reste du sujet en conséquence.

Choix d'une couleur pour un pixel

Comme mentionné précédemment, pour chaque pixel de l'image, vous devez déterminer le nombre d'itérations nécessaires pour déterminer si la suite diverge ou non. Si le nombre d'itérations effectuées est inférieur au nombre maximum d'itérations prévu, on considère que la suite diverge, et le point est colorié en fonction du nombre d'itérations effectuées. Si la suite ne diverge pas, le point est généralement colorié en noir (même si cette règle n'est pas fixe, et que nous pourrons appliquer d'autres règles de coloration dans les prochains *sprints*).

Afin de déterminer la couleur à appliquer à un pixel, vous devez définir une stratégie qui permettra de déterminer, pour un nombre d'itérations donné, la couleur à appliquer au pixel sous la forme d'un objet de type `java.awt.Color`. La première étape va consister à définir une interface pour cette stratégie, définissant la (ou les) méthode(s) appropriée(s) pour cette tâche. Vous veillerez à partager cette interface dès que possible (avant de commencer à l'implémenter), afin que les autres membres

de votre groupe puissent l'utiliser, notamment dans la génération de l'image de fractale présentée plus loin.

Une première implémentation de cette stratégie consiste à colorier les points en niveau de gris, en fonction du ratio r entre le nombre d'itérations effectuées et le nombre maximum d'itérations prévu. La couleur retournée sera celle dont les trois coposantes RGB sont égales à $1 - r$ (ce qui garantit que si r est égal à 1, la couleur retournée est noire).

Dessin de l'image de fractale

Lorsqu'une couleur a été déterminée pour un pixel donné, elle doit ensuite être affectée au pixel correspondant dans l'image, à l'aide de la méthode `void setColor(int row, int column, Color color)` définie dans l'interface `IFractalImage` fournie. Pour implémenter cette interface, vous devez définir un adaptateur pour la classe `java.awt.image.BufferedImage`. Celle-ci vous permet notamment d'affecter une couleur à un pixel de l'image, donné par ses coordonnées (x, y), où le point de coordonnées $(0, 0)$ correspond au pixel en haut à gauche de l'image. **Vous ferez particulièrement attention à l'ordre des coordonnées : lisez bien la documentation de la classe `BufferedImage`.**

Pour sauvegarder une `BufferedImage` dans un fichier, vous pouvez utiliser la méthode `ImageIO.write()` dans votre implémentation de la méthode `saveAs()` de `IFractalImage`.

Génération de l'image de fractale

Pour générer l'image de fractale, il ne reste plus qu'à réunir les classes décrites précédemment, en définissant un générateur de fractale implémentant `IFractalGenerator`. **Vous veillerez à utiliser autant que possible les interfaces et non les classes concrètes, afin de rendre votre implémentation la plus générique possible, et pour ne pas avoir besoin d'attendre les implémentations développées par vos camarades.**

Pour générer une image de fractale, le générateur devra parcourir l'ensemble des pixels de l'image, en utilisant le plan complexe pour déterminer le nombre complexe correspondant à chaque pixel, qui servira de premier terme z_0 pour la suite complexe à utiliser.

Afin de rendre votre implémentation indépendante de la suite considérée, vous devez utiliser dans votre générateur une fonction qui construira la suite complexe à partir de z_0, dont le type sera l'interface fonctionnelle `BiFunction<IComplex, Integer, IComplexSequence>` (vous devrez alors utiliser sa méthode `apply()` avec en paramètre z_0).

La suite créée, il ne reste plus qu'à itérer sur ses termes, et à compter le nombre d'itérations effectuées (rappelez-vous que l'itérateur que vous avez implémenté dans le *sprint* précédent s'arrête « tout seul

» s'il détecte que la suite diverge). Le nombre d'itérations effectuées servira ensuite à déterminer la couleur à l'aide de la stratégie définie plus haut. Il ne reste alors plus qu'à affecter la couleur obtenue au pixel correspondant dans l'image.

Une fois tous les pixels de l'image traités, vous pourrez sauvegarder l'image dans un fichier.

Programme principal

Une fois le générateur de fractale implémenté, vous devez définir une classe principale avec une méthode `main` qui pemettra de générer une image de fractale et de la sauvegarder dans un fichier. Pour l'instant cette méthode `main` doit créer les instances des différents objets nécessaires à la génération de l'image de fractale, en utilisant les implémentations que vous avez définies dans ce *sprint* et les caractéristiques de l'image données plus bas, dans la description de la la démonstration. Notez que pour pouvoir initialiser le générateur de fractale, vous devrez définir une implémentation de `BiFunction<IComplex, Integer, IComplexSequence>` sous la forme d'une expression lambda, qui permettra de créer la suite de Julia à partir du nombre complexe correspondant à chaque pixel de l'image, par exemple :

```
1  (z0, nbTerms) -> new JuliaSequence(z0, new Complex(-0.4, 0.6), nbTerms)
```

Attendus de ce *sprint*

- Implémentation du décorateur permettant de « zoomer » sur le plan complexe.
- Définition d'une stratégie de sélection de couleur à partir d'un nombre d'itérations.
- Implémentation de cette stratégie pour définir une palette en niveaux de gris.
- Définition de l'adaptateur permettant de manipuler une instance de `BufferedImage` en utilisant les méthodes de l'interface `IFractalImage`.
- Implémentation de l'algorithme de génération d'une fractale.
- Définition de la méthode `main` permettant de générer une image de fractale et de la sauvegarder dans un fichier.
- Tests unitaires associés à ces fonctionnalités.

Validations techniques à réaliser en cours de *sprint*

- Implémentation du décorateur de « zoom ».
- Implémentation de la palette de couleurs en niveaux de gris.
- Implémentation de l'adapteur d'`IFractalImage` pour `BufferedImage`.

— Implémentation de l'algorithme de génération.

Démonstration à présenter à la fin du *sprint*

— Une image de taille de 600×800 pixels représentant la fractale de Julia avec $c = -0.4 + 0.6i$ en niveau de gris, avec un nombre maximum d'itérations de 100 et un zoom de 0.004.

Cette image devra être générée pendant la démonstration, dans un fichier dont le nom vous sera donné le moment venu.

Sprint 3 : Variations sur la suite de Julia

Dans ce troisième *sprint*, votre bibliothèque devra permettre de configurer plus finement la génération de l'image de fractale, en proposant de modifier le centre et les couleurs de l'image générée.

But du *sprint*

À l'issue de ce *sprint*, votre bibliothèque doit être capable de générer des images de la fractale de Julia en utilisant différentes palettes de couleurs, et en permettant de zoomer sur une portion du plan complexe qui n'est plus centrée en $(0, 0)$. Vous devrez en particulier pouvoir générer différentes images de la fractale de Julia, dont les aperçus sont donnés ci-dessous (les palettes de couleurs utilisées sont données à titre indicatif, vous serez libres du choix des palettes utilisées lors de la démonstration).

Description technique du *sprint*

Nous allons voir ici qu'il est possible de générer des images de fractales différentes sans pour autant utiliser des suites différentes, mais en modifiant les propriétés de l'image générée.

Modification du centre de l'image

Une première approche consiste à modifier le centre de l'image générée, afin d'observer des détails de la fractale en certains points précis. Pour modifier le centre de l'image, vous devez définir un décorateur pour le plan complexe, de la même manière que vous l'avez fait pour changer d'échelle. Cette fois, le but est de translater les coordonnées du plan complexe, en ajoutant un nombre complexe c à chaque nombre complexe obtenu à partir des coordonnées du pixel.

> **À noter** : si vous appliquez à la fois le décorateur de zoom et le décorateur de translation, veillez à appliquer d'abord le zoom, puis la translation, afin de ne pas modifier le centre de l'image en fonction du zoom appliqué. En effet, si vous appliquez d'abord la translation, puis le zoom, le centre de l'image subira aussi la transformation, et ne sera donc plus le nombre complexe que

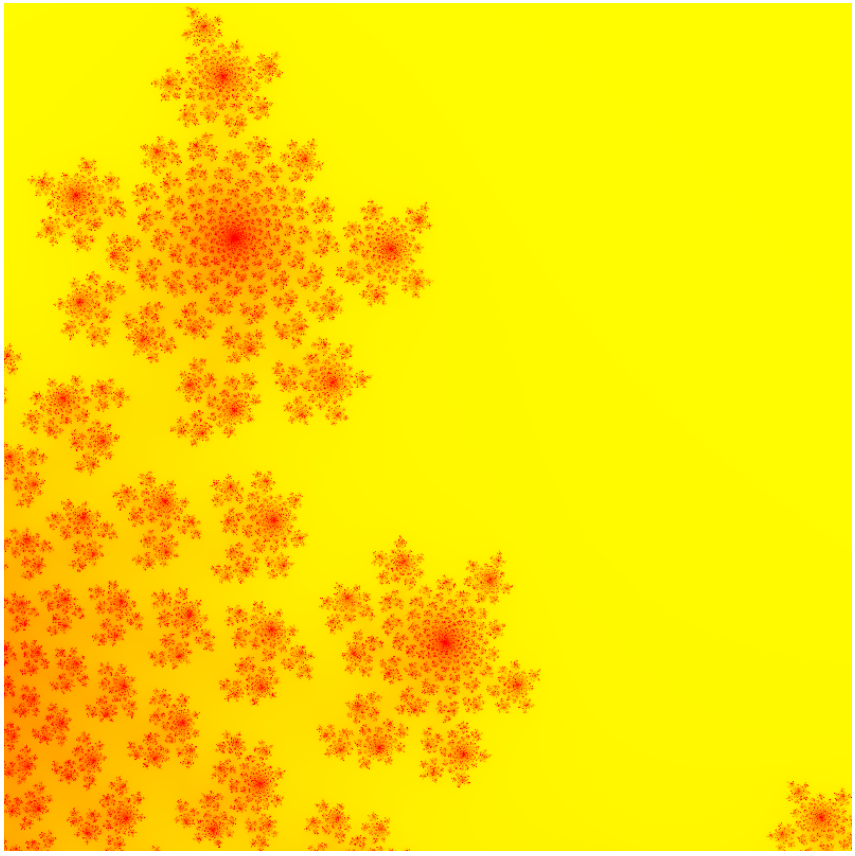


Figure 1 – Fractale de Julia pour $c = -0.4 + 0.6i$ avec un gradient jaune-rouge, centrée en $0.1 + 0.6i$.

> vous aurez choisi.

Changement de la palette de couleurs

Une seconde approche consiste à modifier la palette de couleurs utilisée pour générer l'image de fractale. Il s'agit donc ici d'implémenter de nouvelles stratégies, qui pourront être substituées à celles en niveaux de gris implémentée dans le *sprint* précédent.

Pour générer un niveau d'une couleur primaire (rouge, vert ou bleu), vous devez définir un décorateur qui se chargera de transformer une palette de couleurs existantes en ne conservant que certaines de ces composantes en rouge, vert ou bleu. Pour cela, plusieurs solutions sont possibles : soit vous donnez à votre décorateur trois booléens, qui indiqueront quelle(s) composante(s) conserver, soit vous lui donnez trois nombres réels compris entre 0 et 1, qui indiqueront le pourcentage de chaque composante à conserver. Grâce à ce décorateur, vous pourrez ainsi générer des images en niveaux de rouge, vert ou bleu, en décorant la palette de niveaux de gris (notez qu'il s'agit ici d'un exemple, et que d'autres palettes peuvent être décorées).

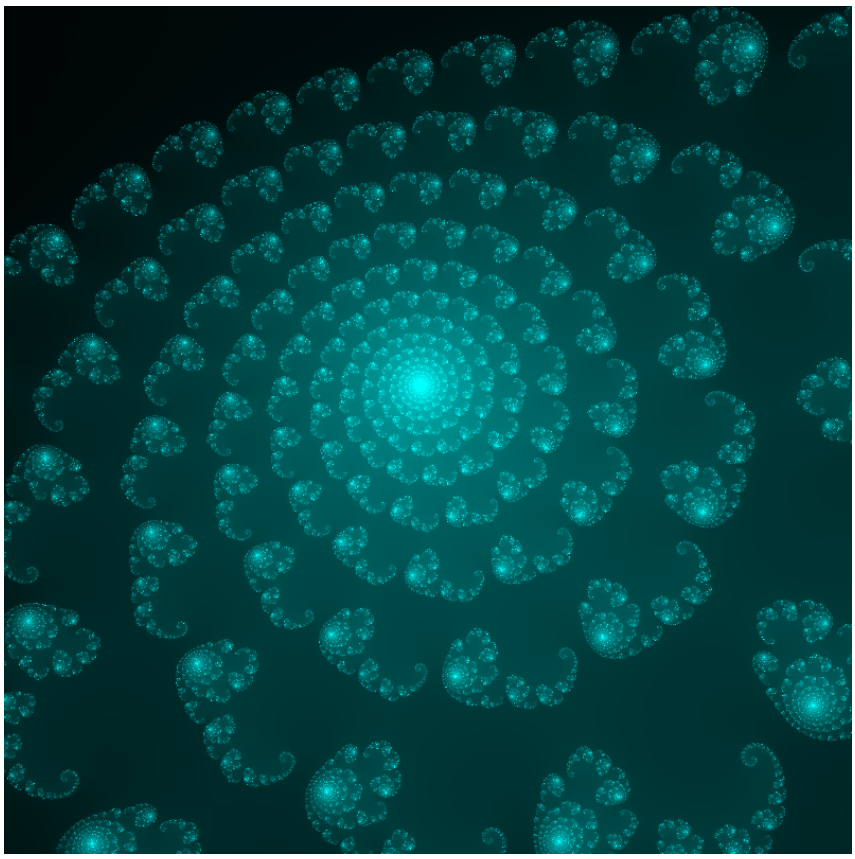


FIGURE 2 – Fractale de Julia pour $c = 0.285 + 0.01i$ avec un niveau de cyan inversé, centrée en $-0.47 - 0.21i$.

Vous pouvez également définir un décorateur qui retournera le complémentaire de la couleur retournée par la palette de couleurs décorée, afin de colorer plutôt les points en lesquels la suite converge et de laisser en noir ceux en lesquels la suite diverge. Pour cela, vous devez créer dans le décorateur une nouvelle couleur dont les composantes sont calculées en soustrayant les composantes de la couleur retournée par la palette décorée à 255.

Une autre stratégie consiste à définir une nouvelle palette qui permettra de générer un dégradé de couleurs entre deux couleurs (r_1, g_1, b_1) et (r_2, g_2, b_2) données. Pour déterminer la couleur à utiliser dans le dégradé, on utilise le ratio p calculé en fonction du nombre d'itérations effectuées par rapport au nombre maximum d'itérations prévu. Les composantes de la couleur à utiliser sont alors calculées de la manière suivante :

$$(r, g, b) = (r_1 + p * (r_2 - r_1), g_1 + p * (g_2 - g_1), b_1 + p * (b_2 - b_1))$$

Enfin, il est aussi possible de définir une palette de couleurs fixe, qui retournera une couleur prédéter-

minée pour chaque nombre d'itérations possible. Dans ce cas, c'est à vous de déterminer les couleurs intermédiaires que vous souhaitez utiliser, et qui vous semblent donner un rendu agréable dans l'image générée. Vous pouvez aussi faire un dégradé entre ces couleurs intermédiaires pour ne pas avoir besoin de définir une couleur pour chaque nombre d'itérations possible, tout en permettant de rendre la transition entre les couleurs plus fluide.

Libre à vous de définir plusieurs palettes fixes, ou de proposer d'autres stratégies en fonction de vos envies et de vos idées. Plus vous proposerez de palettes de couleurs différentes, plus vous pourrez expérimenter différents rendus, et augmenterez vos chances pour le concours d'image en fin de SAÉ.

Attendus de ce *sprint*

— Implémentation du décorateur appliquant une translation sur le plan complexe.
— Implémentation du décorateur permettant de ne conserver que certaines composantes de couleur.
— Implémentation du décorateur permettant d'inverser les couleurs.
— Implémentation d'une palette de couleurs en dégradé.
— Implémentation d'une palette de couleur fixe.
— Tests unitaires associés à ces fonctionnalités.

Validations techniques à réaliser en cours de *sprint*

— Implémentation d'un décorateur appliquant une translation sur le plan complexe.
— Implémentation du décorateur permettant de ne conserver que certaines composantes de couleur.
— Implémentation du décorateur permettant d'inverser les couleurs.
— Implémentation d'une palette de couleurs en dégradé.
— Implémentation d'une palette de couleur fixe.

Démonstration à présenter à la fin du *sprint*

— Une image de taille de 800×800 pixels représentant la fractale de Julia avec $c = -0.4 + 0.6i$, centrée en $0.1 + 0.6i$, avec un nombre maximum d'itérations de 250 et un zoom de 0.001.
— Une image de taille de 800×800 pixels représentant la fractale de Julia avec $c = 0.285 + 0.01i$, centrée en $-0.47 + -0.21i$, avec un nombre maximum d'itérations de 250 et un zoom de 0.001.

Ces images devront être générées pendant la démonstration, dans des fichiers dont le nom vous sera donné le moment venu. Vous pouvez utiliser les palettes de couleurs que vous souhaitez, mais vous utiliserez impérativement deux palettes distinctes, dont l'une utilisera une inversion des couleurs et un « masque » ne conservant que la (les) composante(s) de votre choix.

Sprint 4 : Applications à d'autres suites complexes

Dans ce quatrième *sprint*, votre bibliothèque devra permettre de générer des images de fractales à partir d'autres suites complexes que celle définissant l'ensemble de Julia considéré jusqu'à présent.

But du *sprint*

À l'issue de ce *sprint*, votre bibliothèque doit être capable de générer une image de fractale en considérant la suite de Mandelbrot, des généralisations de la suite de Julia et de la suite de Mandelbrot utilisant des puissances plus élevées que 2, et des suites définies à l'aide d'expressions plus générales. Vous pourrez ainsi représenter des fractales comme celles données ci-dessous.

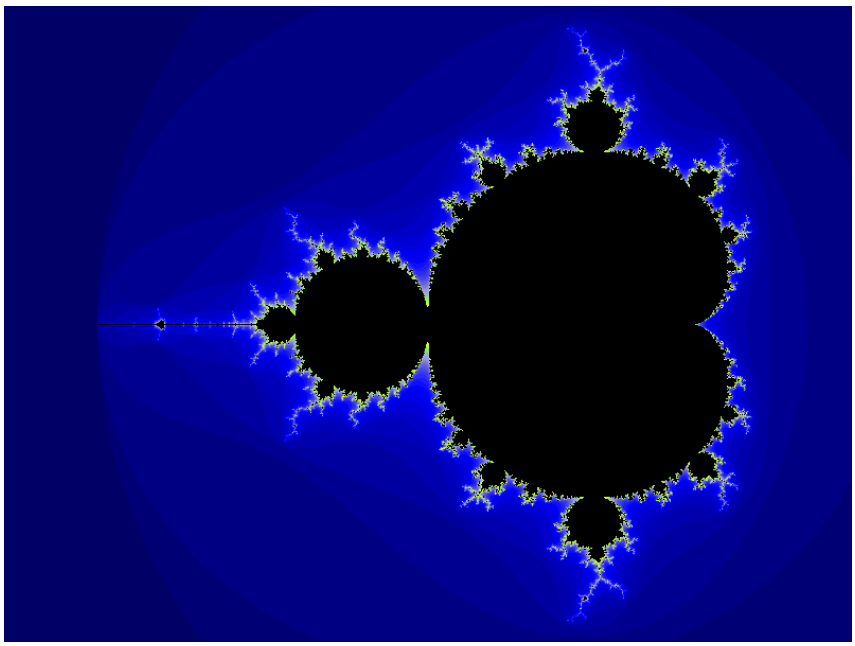


Figure 1 – Fractale de Mandelbrot, centrée en $-0.75 + 0i$.

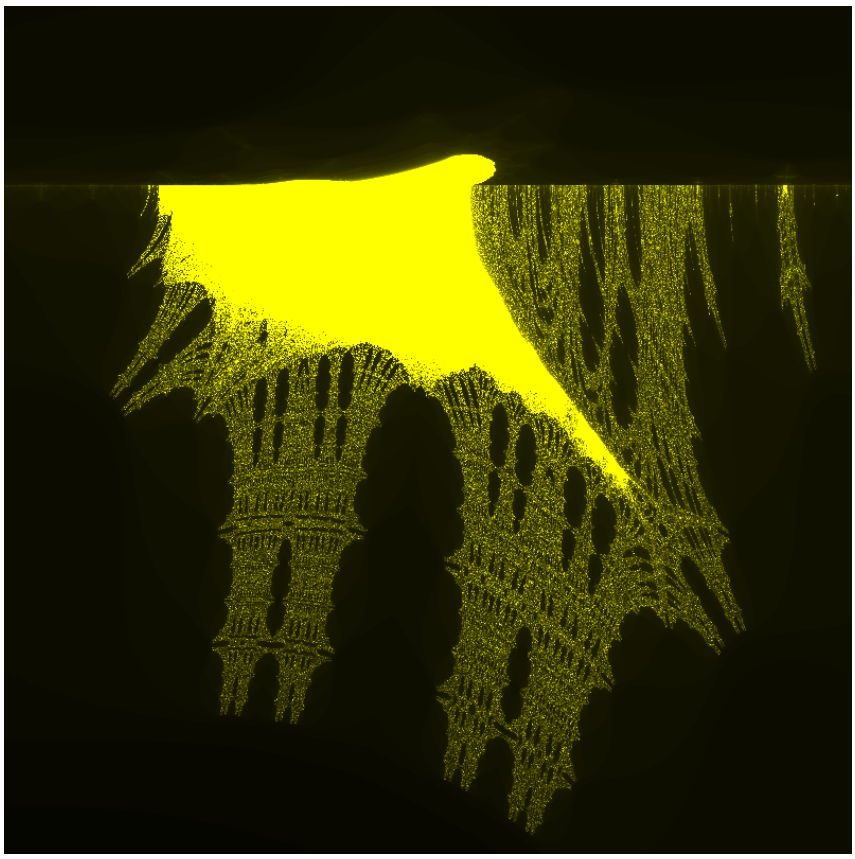


Figure 2 – Fractale du *Burning Ship*, centrée en $-1.75 - 0.03i$.

Description technique du *sprint*

Dans ce *sprint*, vous allez essentiellement devoir fournir d'autres implémentations de l'interface `IComplexSequence`, afin de pouvoir considérer d'autres suites complexes que celle définissant l'ensemble de Julia.

Suite de Mandelbrot

La première suite que vous allez implémenter est la suite de Mandelbrot, qui est définie à partir de la même expression que celle utilisée pour l'ensemble de Julia, sauf que le nombre c est remplacé par le nombre complexe pour lequel la suite est évaluée :

- $m_n(z) = m_{n-1}(z)^2 + z$

Suites Multijulia et Multibrot

D'autres variantes consistent à utiliser une puissance p plus élevée que 2 dans l'expression de la suite, pour obtenir des généralisations de l'ensemble de Julia et de l'ensemble de Mandelbrot qui comportent alors plus de « lobes » :

- $j_n(z) = j_{n-1}(z)^p + c$
- $m_n(z) = m_{n-1}(z)^p + z$

Pour implémenter ces suites, appelées respectivement *Multijulia* et *Multibrot*, il vous sera plus simple d'ajouter l'opération de puissance dans l'interface `IComplex`, et l'implémenter dans la classe `Complex`. Notez que cette opération sur les complexes fonctionne de la même manière que sur les réels, en répétant p fois l'opération de multiplication sur le nombre complexe considéré.

Suites du *Burning Ship* et du *Tricorn*

La suite *Burning Ship* est une autre suite complexe qui peut être utilisée pour générer une fractale, dont la formule consiste à modifier la formule de la suite de Julia en prenant la valeur absolue de la partie réelle et de la partie imaginaire du nombre complexe obtenu à l'étape précédente, avant d'élever au carré le nombre complexe obtenu :

- $b_n(z) = (|\mathfrak{Re}(b_{n-1}(z))| + i|\mathfrak{Im}(b_{n-1}(z))|)^2 + c$

La suite *Tricorn* (ou *Mandelbar*) est encore une autre suite complexe qui peut être utilisée pour générer une fractale, dont la formule consiste à modifier la formule de la suite de Julia en prenant le conjugué du nombre complexe obtenu à l'étape précédente, avant de l'élever au carré :

- $t_n(z) = \overline{(t_{n-1}(z))}^2 + c$

Les mêmes généralisations que pour les suites de Julia et de Mandelbrot peuvent être appliquées aux suites *Burning Ship* et *Tricorn*.

Suites définies à l'aide d'expressions plus générales

Vous pouvez également implémenter d'autres suites complexes, en utilisant des expressions plus générales que celles présentées ci-dessus, par exemple en utilisant des fonctions trigonométriques ou exponentielles, ou en utilisant des polynômes ayant plus de deux termes. Laissez libre cours à votre imagination pour proposer des suites complexes intéressantes. Vous pouvez également demander à l'IA de vous proposer des expressions de suites complexes susceptibles de générer de belles fractales.

Pour éviter de créer une nouvelle classe à chaque fois, vous devez utiliser l'interface fonctionnelle `BinaryOperator<IComplex>` pour définir à l'aide d'une expression lambda la fonction f carac-

téristique de la suite étudiée (ses paramètres sont *a priori* le premier terme de la suite z_0 et la valeur précédente z). Cette fonction sera alors utilisée dans une implémentation générique de l'interface `IComplexSequence`, qui pourra ainsi être utilisée pour générer des suites complexes à partir de n'importe quelle expression. Si vous n'avez pas encore implémenté les autres suites lorsque vous lisez ces lignes, notez qu'il est tout à fait possible de commencer par implémenter cette suite générique, et de l'utiliser ensuite pour instancier les autres suites complexes suggérées ci-dessus. Une seule classe peut donc suffire pour implémenter ce *sprint*, mais vous veillerez dans tous les cas à permettre de facilement pouvoir instancier toutes les suites demandées ici (par exemple, à l'aide de fabriques statiques).

Attendus de ce *sprint*

— Implémentation de la suite de Mandelbrot.
— Implémentation de la suite de Julia généralisée.
— Implémentation de la suite de Mandelbrot généralisée.
— Implémentation de la suite *Burning Ship*.
— Implémentation de la suite *Burning Ship* généralisée.
— Implémentation de la suite *Tricorn*.
— Implémentation de la suite *Tricorn* généralisée.
— Implémentation d'au moins une suite complexe définie à l'aide d'une expression plus générale utilisant une expression lambda.
— Tests unitaires associés à ces fonctionnalités.

Validations techniques à réaliser en cours de *sprint*

— Implémentation de la suite de Julia généralisée.
— Implémentation de la suite de Mandelbrot généralisée.
— Utilisation des expressions lambda pour les suites générales.

Démonstration à présenter à la fin du *sprint*

— Une image de taille 600×800 pixels représentant la fractale de Mandelbrot, centrée en $-0.75+0i$ avec un nombre maximum d'itérations de 250 et un zoom de 0.001.

— Une image de taille 800×800 pixels représentant la fractale du *Burning Ship*, centrée en $-1.755 - 0.03i$ avec un nombre maximum d'itérations de 64 et un zoom de 0.0001, utilisant $c = -0.4 + 0.6i$.
— Une image au choix d'une suite complexe définie à l'aide d'une expression lambda.

Ces images devront être générées pendant la démonstration, dans des fichiers dont le nom vous sera donné le moment venu.

Sprint 5 : Configuration du générateur

Dans ce cinquième *sprint*, vous allez devoir implémenter un constructeur (*builder*) pour configurer votre générateur de fractales, afin de fournir une manière simple de configurer la génération d'une image à partir de ses propriétés.

But du *sprint*

À l'issue de ce *sprint*, votre bibliothèque devra être capable de construire un générateur de fractales à partir d'une configuration donnée au travers d'un constructeur (*builder*), et de générer une image de fractale à partir de ce générateur. Le *builder* devra notamment permettre de spécifier la suite à utiliser, la taille de l'image, le zoom initial, le point central de l'image, le nombre d'itérations à réaliser, la palette de couleurs utilisée et le nom du fichier de sortie. À la fin du *sprint* vous devrez générer trois fractales au choix parmi celles proposées depuis le début de la SAÉ, en montrant comment votre *builder* permet de spécifier les paramètres de génération de ces fractales.

Description technique du *sprint*

Afin de simplifier la configuration du générateur de fractales, vous allez devoir implémenter un *builder* qui utilisera un certain nombre d'énumérations pour représenter différentes fonctionnalités, afin de simplifier son usage.

Encapsulation des suiteset stratégies dans des énumérations

Depuis le début de la SAÉ, vous avez défini un certain nombre de suites définies dans des classes différentes, dont les instanciations sont différentes (avec une constante ou non, avec une expression lambda ou non, *etc.*). De manière à uniformiser l'utilisation de ces suites, il es possible de les encapsuler dans une énumération. Chaque constante correspondra alors à une suite (ce qui aura en plus l'avantage de la nommer explicitement), et sera associée à une expression lambda permettant de créer

l'instance de la suite correspondante, en utilisant l'interface fonctionnelle `BiFunction<IComplex, Integer, IComplexSequence>` que vous avez déjà utilisée dans les *sprints* précédents. Vous devrez fournir une méthode dans l'énumération permettant d'obtenir cette fonction, ou de créer la suite correspondante en utilisant la fonction (à vous de choisir ce que vous préférez).

De la même manière, vous devez définir une énumération pour encapsuler vos différentes stratégies de palettes de couleurs, afin de pouvoir les sélectionner plus facilement en les nommant. Chaque instance devra donc être associée à une palette de couleurs qui pourra être obtenue au travers d'un appel de méthode. En fonction de la manière dont vous avez défini votre stratégie, il vous faudra peut-être paramétrer cette méthode par la taille de la palette, et donc adapter le type de l'attribut de l'énumération en conséquence.

Implémentation du *builder*

Le *builder* que vous allez implémenter devra permettre de configurer tous les paramètres de génération d'une fractale : la suite à utiliser, la taille de l'image, le zoom initial, le point central de l'image, le nombre d'itérations à réaliser, la palette de couleurs utilisée et le nom du fichier de sortie. Vous utiliserez pour cela la « variante » de ce patron présentée dans le cours.

Pour la suite et la palette de couleurs, vous devrez permettre à la fois d'utiliser les énumérations que vous avez définies, mais également de pouvoir fournir directement les objets « bruts », afin de permettre de conserver la flexibilité permise par l'utilisation des patrons de conception.

Pour chaque méthode que vous définirez dans le *builder*, vous veillerez à ce qu'elle retourne l'instance du *builder* elle-même, afin de permettre d'enchaîner les appels de méthodes pour configurer le générateur. Vous choisirez également judicieusement les noms des méthodes, afin de rendre l'utilisation du *builder* la plus claire possible : lire les appels de méthodes chaînés doit fournir une description de la fractale qui sera générée, presque en langage naturel. Par ailleurs, veillez à ne pas dupliquer les paramètres de configuration dans le *builder* : par exemple, la hauteur et la largeur de l'image permettent à la fois de créer l'image et le plan complexe, tandis que le nombre d'itérations a une influence sur la palette de couleurs. Assurez-vous donc de mettre à jour tous les éléments qui dépendent d'un paramètre de configuration lorsque celui-ci est modifié.

Vous utiliserez enfin ce *builder* pour initialiser le générateur de fractales dans la méthode `main`, en respectant le fonctionnement de ce patron de conception.

Attendus de ce *sprint*

— Définition d'une énumération pour les palettes de couleurs.

— Définition d'une énumération pour les suites complexes.
— Implémentation du *builder*.
— Mise à jour de la méthode `main` pour utiliser le *builder*.
— Tests unitaires associés à ces fonctionnalités.

Validations techniques à réaliser en cours de *sprint*

— Définition d'une énumération pour les palettes de couleurs.
— Définition d'une énumération pour les suites complexes.
— Implémentation du *builder*.
— Mise à jour de la méthode `main` pour utiliser le *builder*.

Démonstration à présenter à la fin du *sprint*

— Génération des trois images que vous choisirez, en modifiant le `main` pendant la démonstration.

Ces images devront être générées pendant la démonstration, dans des fichiers dont le nom vous sera donné le moment venu.

Sprint 6 : Génération d'une animation

Dans ce sixième *sprint*, votre bibliothèque devra permettre de générer des images GIF animées d'une fractale, en permettant de zoomer ou de passer d'une fractale à une autre en appliquant une transformation.

But du *sprint*

À l'issue de ce *sprint*, votre bibliothèque devra permettre de sauvegarder une image GIF animée d'une fractale, notamment en zoomant sur un point du plan complexe, ou en passant d'une fractale à une autre en appliquant une transformation sur les suites considérées.

Description technique du *sprint*

Dans ce *sprint*, vous allez devoir implémenter un composite pour l'interface `IFractalGenerator`, qui permettra de générer plusieurs images de fractales successives, qui seront ensuite collectées pour former une image GIF animée. Pour cela, vous devez définir une nouvelle classe implémentant `IFractalGenerator`, qui contiendra elle-même une liste de `IFractalGenerator`. La génération d'une image dans ce composite consistera à déléguer l'opération à la totalité des générateurs contenus dans la liste, et à collecter les images générées pour les sauvegarder dans un fichier GIF animé. Pour cela, vous pouvez utiliser la bibliothèque `com.madgag:animated-gif-lib` et l'ajouter à votre projet en ajoutant la dépendance suivante dans le fichier `build.gradle` :

```
dependencies {
    implementation 'com.madgag:animated-gif-lib:1.4'
}
```

Une fois la dépendance ajoutée et le projet rafraîchi, vous pourrez utiliser la classe `AnimatedGifEncoder` pour générer l'image GIF animée dans la méthode `saveImage()` de votre composite, en prenant exemple sur l'extrait de code suivant :

```
1  AnimatedGifEncoder encoder = new AnimatedGifEncoder();
2  encoder.start(outputPath);
3  encoder.setDelay(delayMs); // Faites des essais pour trouver le bon dé
       lai.
4  encoder.setRepeat(0);
5  encoder.setQuality(10);
6
7  for (BufferedImage image : images) {
8      encoder.addFrame(image);
9  }
10
11 encoder.finish();
```

> **À noter** : vous n'avez *a priori* pas accès à l'instance de `BufferedImage` générée par le générateur de fractales, car elle est encapsulée dans l'adaptateur que vous avez implémenté dans un *sprint* précédent. Une manière simple de la récupérer est de modifier l'interface `IFractalImage` pour ajouter par exemple une méthode `asBufferedImage()` qui permettra de récupérer l'image adaptée dans le cas de l'adaptateur.

Pour créer les instances de `IFractalGenerator` vous allez exploiter le *builder* que vous avez implémenté dans le *sprint* précédent, en créant successivement plusieurs instances de générateurs de fractales, en modifiant à chaque fois certaines propriétés de l'image générée tout en conservant les autres propriétés identiques. Pour cette création, vous avez plusieurs options possible : par exemple, vous pouvez définir un *builder* composite (ce qui demandera d'ajouter une interface pour votre *builder*), ou une fabrique abstraite, ou une fabrique statique. D'autres solutions peuvent aussi envisagées : à vous de choisir celle qui vous semble la plus adaptée, **en veillant à respecter les bonnes pratiques de la programmation orientée objet**.

Dans le cas d'une animation consistant à zoomer sur un point du plan complexe, chaque instance va avoir un zoom plus important que la précédente. Vous devrez donc multiplier le zoom précédent par une valeur comprise entre 0 et 1 pour chaque nouveau générateur, afin de réduire la portion du plan complexe représentée par l'image. À vous d'expérimenter différentes valeurs pour obtenir une animation la plus fluide possible.

> **Attention cependant !**
>
> Vous ne pourrez pas zoomer à l'infini, car les **`double`** n'ont qu'une précision limitée : au bout d'un certain nombre d'images, votre image va perdre en netteté, en raison des erreurs d'arrondi qui vont s'accumuler. Là-aussi, vous devrez expérimenter différentes valeurs déterminer le nombre

> d'images maximum que vous pourrez générer avant que l'image ne devienne floue. (ou que la mémoire ne soit saturée, si vous générez trop d'images ou de trop grandes images).

Une autre option consiste à générer une animation permettant de passer d'une fractale à une autre, en appliquant une transformation sur la suite considérée. Dans le cas de la suite de Julia, vous pouvez ainsi générer une animation en appliquant une rotation sur la constante c utilisée dans la suite. Plus précisément, vous pouvez multiplier la constante c par le nombre complexe $\cos(\theta) + i\sin(\theta)$, où θ varie entre 0 et 2π pour passer d'une suite à une autre. Comme pour le zoom, vous devrez expérimenter différentes valeurs pour déterminer le « pas » entre deux angles θ successifs, afin d'obtenir une animation la plus fluide possible. La fractale donnera alors l'impression de tourner sur elle-même, tout en changeant de forme.

> **À noter** : pour ce cas particulier, vous pouvez soit choisir de restreindre la rotation à une ou plusieurs suites particulières, soit ajouter une méthode dans l'interface `IComplexSequence` pour permettre de transformer la suite considérée, et donc de générer une animation pour n'importe quelle suite complexe (même si, dans certains cas, une telle transformation n'aurait pas de sens et devrait jeter une `UnsupportedOperationException`). La seconde option est sans doute la plus flexible, mais elle est aussi plus complexe à implémenter. À vous de choisir comment vous souhaitez implémenter cette fonctionnalité, en fonction de votre avancement.

Vous pouvez générer de nombreuses autres animations, en appliquant par exemple la rotation précédente à d'autres suites complexes, ou en modifiant d'autres caractéristiques de l'image ou la palette de couleurs utilisée. Laissez libre court à votre imagination pour créer des animations originales, qui pourront vous avantager lors du concours proposé en fin de semaine.

Attendus de ce *sprint*

- Implémentation d'un composite de `IFractalGenerator` permettant de générer plusieurs images de fractales successives et de les assembler.
- Implémentation du zoom.
- Implémentation de la rotation.
- Ajout des méthodes dans le *builder* pour configurer le composite.
- Tests unitaires associés à ces fonctionnalités.

Validations techniques à réaliser en cours de *sprint*

- Structure du composite.

— Instanciation des générateurs de fractales pour le zoom.
— Instanciation des générateurs de fractales pour la rotation.

Démonstration à présenter à la fin du *sprint*

— Une première animation avec la fractale de Mandelbrot, en zoomant sur le point $-0.743643887037151 + 0.131825904205330i$ (qui sera donc aussi le centre de l'image).
— Une seconde animation avec la fractale de Julia centrée en 0, avec pour valeur initiale $c = 0.7885 + 0i$ à laquelle vous appliquerez la rotation.

Libre à vous de configurer les autres critères comme bon vous semble. **Attention : ne générez pas des images trop grandes pour votre animation, car la génération pourrait être à la fois très longue et coûteuse en mémoire.**

Sprint 7 : Gestion de la ligne de commande et finalisation du projet

Pour ce septième et dernier *sprint*, l'objectif est d'implémenter une interface en ligne de commande (CLI) pour permettre aux utilisateurs d'exécuter votre application tout en configurant comment la fractale doit être générée via des options données à votre programme.

But du *sprint*

À l'issue de ce *sprint*, vous devrez démontrer le bon fonctionnement de votre application en générant les images des différentes fractales demandées au cours de cette SAÉ **uniquement en utilisant la ligne de commande**. Cette dernière devra être utilisée pour préciser tous les paramètres de génération de l'image, tels que le nom de la fractale, la taille de l'image, le zoom initial, le point central, le nombre d'itérations à réaliser, la palette de couleurs utilisée, le type d'image à générer (image unique, zoom progressif ou rotation) et le nom du fichier de sortie.

Vous devrez également vous assurer que votre application respecte les bonnes pratiques de la programmation, en utilisant l'outil d'analyse statique *SonarQube* et en vous assurant que la couverture de code de vos tests unitaires est d'au moins 80%.

Description technique du *sprint*

Ce *sprint* vise à permettre la diffusion de votre application et à la rendre utilisable par un utilisateur final, en lui permettant de configurer la génération de l'image de fractale à l'aide d'une interface en ligne de commande.

Gestion de la ligne de commande

Pour interpréter les arguments de la ligne de commande, vous devez choisir une bibliothèque vous ppermettant de gérer les options et les arguments passés à votre application. **Vous ne devez pas traiter**

directement les arguments vous-mêmes. Vous ferez ce choix en début de *sprint*, et vous l'utiliserez pour implémenter la gestion de la ligne de commande dans votre application **après validation par un enseignant**.

Les options suivantes doivent impérativement être gérées par la ligne de commande (vous pourrez en ajouter d'autres si vous le souhaitez, notamment pour supporter des fonctionnalités supplémentaires que vous auriez implémentées) :

- `-f/--fractal-name <nom>` : le nom de la fractale à générer (vous utiliserez le nom défini dans l'énumération correspondante).
- `-n/--nb-iterations <nombre>` : le nombre d'itérations maximal à appliquer pour générer la fractale.
- `-h/--height <pixels>` : la hauteur de l'image à générer en pixels.
- `-w/--width <pixels>` : la largeur de l'image à générer en pixels.
- `-s/--scale <zoom>` : l'échelle à appliquer sur l'image.
- `-x/--focus-x <nombre>` : la partie réelle du point central de l'image.
- `-y/--focus-y <nombre>` : la partie imaginaire du point central de l'image.
- `-p/--palette-name <nom>` : le nom de la palette de couleurs à appliquer lors de la génération (vous utiliserez le nom défini dans l'énumération correspondante).
- `-t/--type <type>` : le type d'image à générer, qui peut être soit une image unique, soit un zoom progressif, soit une rotation.
- `-i/--image-count <nombre>` : le nombre d'images à générer pour les animations.
- `-o/--output <fichier>` : le nom du fichier dans lequel la fractale doit être sauvegardée.

Vous adapterez votre classe principale de manière à ce que le générateur de fractale soit configuré à partir des paramètres passés dans la ligne de commande, en utilisant le *builder* définit lors d'un précédent *sprint*.

En cas de mauvais usage de la ligne de commande, ou si aucun paramètre n'est fourni, votre application doit afficher un message d'aide reprenant les descriptions de toutes les options disponibles, **en anglais**. Vous compléterez également votre fichier `README.md` pour expliquer comment utiliser votre application en ligne de commande.

Bonnes pratiques de programmation

Vous profiterez également de ce *sprint* pour vérifier la qualité de votre projet, en effectuant une analyse statique du code sur l'instance *SonarQube* de l'université Pour cela, vous devez d'abord définir un *token* d'authentification depuis l'onglet `Sécurité` des paramètres de votre compte sur *SonarQube*, auquel vous pouvez accéder en vous connectant via l'authentification par *GitLab*.

Vous devez aussi configurer votre projet pour qu'il soit analysé par *SonarQube* dans votre fichier

`build.gradle`, qui comporte déjà le plugin *SonarQube*. Vous devez uniquement mettre à jour la ligne suivante, en précisant un nom de projet qui l'identifie de manière unique par rapport aux autres groupes (par exemple, en ajoutant vos initiales à la fin du nom du projet) :

```
property("sonar.projectKey", "fractalens")
```

Pour effectuer l'analyse *SonarQube*, il suffit ensuite d'exécuter la commande suivante :

```
./gradlew sonar -Dsonar.token=letokendefiniplustot
```

Le résultat de l'analyse sera visible sur un lien qui sera affiché à l'issue de l'exécution de la commande précédente, et qui vous permettra de vérifier la qualité de votre code et de corriger les problèmes éventuels.

Vous compléterez également vos tests unitaires afin d'atteindre un seuil de couverture de code d'au moins 80%.

Attendus de ce *sprint*

- Gestion de la ligne de commande pour interagir avec l'application.
- Démonstration complète de votre application, avec une ligne de commande fonctionnelle et une image générée correctement à partir de la configuration donnée.
- Finalisation de l'application, avec une couverture de code d'au moins 80% et une analyse statique du code sans problèmes majeurs.

Validation technique à réaliser en cours de *sprint*

- Choix et utilisation de la bibliothèque de gestion de la ligne de commande.
- Aide de la ligne de commande.
- Résultats de l'analyse *SonarQube*.
- Couverture des tests unitaires à hauteur de 80% minimum.
- Javadoc complète en anglais.

Démonstration à présenter à la fin du *sprint*

- Pour chaque image générée au cours de cette SAÉ, vous devez être capable de montrer que vous pouvez la générer uniquement en utilisant la ligne de commande, avec les paramètres appropriés, **sans modifier le code de votre application** entre deux exécutions.

Final de la SAÉ - Concours de fractales

Pour la dernière étape de la SAÉ, nous vous proposons de participer à un concours de génération de fractales, dans lequel vous devrez utiliser votre bibliothèque pour générer différentes images, et les soumettre au vote pour déterminer les plus belles d'entre elles. Chaque groupe devra déposer sur *Moodle* trois images, dans trois catégories différentes :

— Catégorie 1 : Zoom sur un point au choix de la fractale de Mandelbrot
— Catégorie 2 : Rotation de la suite de Julia pour une constante c au choix
— Catégorie 3 : Fractale à partir d'une suite complexe libre, animée ou non

Les images anonymisées seront ensuite mises à disposition de toutes et tous pour que vous puissiez les visualiser et voter pour vos images préférées.

Règles de soumission

Vous devez respecter les règles suivantes pour chaque image déposée :

— L'image doit être générée uniquement **à partir de votre application**, que vous pouvez néanmoins adapter en tenant compte des règles ci-après.
— Vous **ne pouvez pas** utiliser des images trouvées en ligne ou générées par d'autres applications ou par IA, ni retoucher vos images à l'issue de la génération. Toute image ne respectant pas cette règle sera disqualifiée.
— Vous **pouvez** modifier votre bibliothèque pour améliorer le rendu de vos images, notamment pour supporter d'autres suites complexes ou d'autres palettes de couleurs.
— Vous pouvez choisir librement la palette de couleurs utilisée pour vos images, ainsi que la vitesse du zoom et le nombre d'images générées pour l'animation.

Règles du vote

Une fois les images mises à disposition, vous pourrez les visualiser et voter pour vos images préférées dans chaque catégorie en suivant les règles suivantes :

— La méthode de vote utilisée sera le vote par **approbation** : vous pouvez voter pour autant d'images que vous le souhaitez dans chaque catégorie (cette méthode de vote est moins sensible aux manipulations).
— Le vote est **individuel** et **anonyme** : chacun.e d'entre vous peut voter pour ses images préférées dans chaque catégorie, sans que vous ayez besoin de vous mettre d'accord avec les autres membres de votre groupe.

Les résultats seront annoncés à l'issue du vote dans une salle qui vous sera communiquée en temps voulu.